

Chapter 13

Linear Factor Models

Many of the research frontiers in deep learning involve building a probabilistic model of the input, $p_{\text{model}}(\mathbf{x})$. Such a model can, in principle, use probabilistic inference to predict any of the variables in its environment given any of the other variables. Many of these models also have latent variables $\mathbf{h}$, with $p_{\text{model}}(\mathbf{x}) = \mathbb{E}_{\mathbf{h}} p_{\text{model}}(\mathbf{x} | \mathbf{h})$. These latent variables provide another means of representing the data. Distributed representations based on latent variables can obtain all of the advantages of representation learning that we have seen with deep feedforward and recurrent networks.

In this chapter, we describe some of the simplest probabilistic models with latent variables: linear factor models. These models are sometimes used as building blocks of mixture models (Hinton *et al.*, 1995a; Ghahramani and Hinton, 1996; Roweis *et al.*, 2002) or larger, deep probabilistic models (Tang *et al.*, 2012). They also show many of the basic approaches necessary to build generative models that the more advanced deep models will extend further.

A linear factor model is defined by the use of a stochastic, linear decoder function that generates $\mathbf{x}$ by adding noise to a linear transformation of $\mathbf{h}$.

These models are interesting because they allow us to discover explanatory factors that have a simple joint distribution. The simplicity of using a linear decoder made these models some of the first latent variable models to be extensively studied.

A linear factor model describes the data generation process as follows. First, we sample the explanatory factors $\mathbf{h}$ from a distribution

$$\mathbf{h} \sim p(\mathbf{h}), \tag{13.1}$$

where $p(\mathbf{h})$ is a factorial distribution, with $p(\mathbf{h}) = \prod_i p(h_i)$, so that it is easy to

sample from. Next we sample the real-valued observable variables given the factors:

$$\mathbf{x} = \mathbf{W}\mathbf{h} + \mathbf{b} + \text{noise} \quad (13.2)$$

where the noise is typically Gaussian and diagonal (independent across dimensions). This is illustrated in figure 13.1.

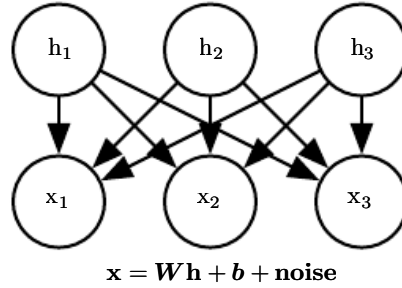


Figure 13.1: The directed graphical model describing the linear factor model family, in which we assume that an observed data vector $\mathbf{x}$ is obtained by a linear combination of independent latent factors $\mathbf{h}$, plus some noise. Different models, such as probabilistic PCA, factor analysis or ICA, make different choices about the form of the noise and of the prior $p(\mathbf{h})$.

13.1 Probabilistic PCA and Factor Analysis

Probabilistic PCA (principal components analysis), factor analysis and other linear factor models are special cases of the above equations (13.1 and 13.2) and only differ in the choices made for the noise distribution and the model's prior over latent variables $\mathbf{h}$ before observing $\mathbf{x}$.

In **factor analysis** (Bartholomew, 1987; Basilevsky, 1994), the latent variable prior is just the unit variance Gaussian

$$\mathbf{h} \sim \mathcal{N}(\mathbf{h}; \mathbf{0}, \mathbf{I}) \quad (13.3)$$

while the observed variables x_i are assumed to be **conditionally independent**, given $\mathbf{h}$. Specifically, the noise is assumed to be drawn from a diagonal covariance Gaussian distribution, with covariance matrix $\boldsymbol{\psi} = \text{diag}(\boldsymbol{\sigma}^2)$, with $\boldsymbol{\sigma}^2 = [\sigma_1^2, \sigma_2^2, \dots, \sigma_n^2]^\top$ a vector of per-variable variances.

The role of the latent variables is thus to *capture the dependencies* between the different observed variables x_i . Indeed, it can easily be shown that $\mathbf{x}$ is just a multivariate normal random variable, with

$$\mathbf{x} \sim \mathcal{N}(\mathbf{x}; \mathbf{b}, \mathbf{W}\mathbf{W}^\top + \boldsymbol{\psi}). \quad (13.4)$$

In order to cast PCA in a probabilistic framework, we can make a slight modification to the factor analysis model, making the conditional variances σ_i^2 equal to each other. In that case the covariance of $\mathbf{x}$ is just $\mathbf{W}\mathbf{W}^\top + \sigma^2\mathbf{I}$, where σ^2 is now a scalar. This yields the conditional distribution

$$\mathbf{x} \sim \mathcal{N}(\mathbf{x}; \mathbf{b}, \mathbf{W}\mathbf{W}^\top + \sigma^2\mathbf{I}) \quad (13.5)$$

or equivalently

$$\mathbf{x} = \mathbf{W}\mathbf{h} + \mathbf{b} + \sigma\mathbf{z} \quad (13.6)$$

where $\mathbf{z} \sim \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$ is Gaussian noise. [Tipping and Bishop \(1999\)](#) then show an iterative EM algorithm for estimating the parameters $\mathbf{W}$ and σ^2 .

This **probabilistic PCA** model takes advantage of the observation that most variations in the data can be captured by the latent variables $\mathbf{h}$, up to some small residual **reconstruction error** σ^2 . As shown by [Tipping and Bishop \(1999\)](#), probabilistic PCA becomes PCA as $\sigma \rightarrow 0$. In that case, the conditional expected value of $\mathbf{h}$ given $\mathbf{x}$ becomes an orthogonal projection of $\mathbf{x} - \mathbf{b}$ onto the space spanned by the d columns of $\mathbf{W}$, like in PCA.

As $\sigma \rightarrow 0$, the density model defined by probabilistic PCA becomes very sharp around these d dimensions spanned by the columns of $\mathbf{W}$. This can make the model assign very low likelihood to the data if the data does not actually cluster near a hyperplane.

13.2 Independent Component Analysis (ICA)

Independent component analysis (ICA) is among the oldest representation learning algorithms ([Herault and Ans, 1984](#); [Jutten and Herault, 1991](#); [Comon, 1994](#); [Hyvärinen, 1999](#); [Hyvärinen *et al.*, 2001a](#); [Hinton *et al.*, 2001](#); [Teh *et al.*, 2003](#)). It is an approach to modeling linear factors that seeks to separate an observed signal into many underlying signals that are scaled and added together to form the observed data. These signals are intended to be fully independent, rather than merely decorrelated from each other.¹

Many different specific methodologies are referred to as ICA. The variant that is most similar to the other generative models we have described here is a variant ([Pham *et al.*, 1992](#)) that trains a fully parametric generative model. The prior distribution over the underlying factors, $p(\mathbf{h})$, must be fixed ahead of time by the user. The model then deterministically generates $\mathbf{x} = \mathbf{W}\mathbf{h}$. We can perform a

¹See section 3.8 for a discussion of the difference between uncorrelated variables and independent variables.

nonlinear change of variables (using equation 3.47) to determine $p(\mathbf{x})$. Learning the model then proceeds as usual, using maximum likelihood.

The motivation for this approach is that by choosing $p(\mathbf{h})$ to be independent, we can recover underlying factors that are as close as possible to independent. This is commonly used, not to capture high-level abstract causal factors, but to recover low-level signals that have been mixed together. In this setting, each training example is one moment in time, each x_i is one sensor's observation of the mixed signals, and each h_i is one estimate of one of the original signals. For example, we might have n people speaking simultaneously. If we have n different microphones placed in different locations, ICA can detect the changes in the volume between each speaker as heard by each microphone, and separate the signals so that each h_i contains only one person speaking clearly. This is commonly used in neuroscience for electroencephalography, a technology for recording electrical signals originating in the brain. Many electrode sensors placed on the subject's head are used to measure many electrical signals coming from the body. The experimenter is typically only interested in signals from the brain, but signals from the subject's heart and eyes are strong enough to confound measurements taken at the subject's scalp. The signals arrive at the electrodes mixed together, so ICA is necessary to separate the electrical signature of the heart from the signals originating in the brain, and to separate signals in different brain regions from each other.

As mentioned before, many variants of ICA are possible. Some add some noise in the generation of $\mathbf{x}$ rather than using a deterministic decoder. Most do not use the maximum likelihood criterion, but instead aim to make the elements of $\mathbf{h} = \mathbf{W}^{-1}\mathbf{x}$ independent from each other. Many criteria that accomplish this goal are possible. Equation 3.47 requires taking the determinant of $\mathbf{W}$, which can be an expensive and numerically unstable operation. Some variants of ICA avoid this problematic operation by constraining $\mathbf{W}$ to be orthogonal.

All variants of ICA require that $p(\mathbf{h})$ be non-Gaussian. This is because if $p(\mathbf{h})$ is an independent prior with Gaussian components, then $\mathbf{W}$ is not identifiable. We can obtain the same distribution over $p(\mathbf{x})$ for many values of $\mathbf{W}$. This is very different from other linear factor models like probabilistic PCA and factor analysis, that often require $p(\mathbf{h})$ to be Gaussian in order to make many operations on the model have closed form solutions. In the maximum likelihood approach where the user explicitly specifies the distribution, a typical choice is to use $p(h_i) = \frac{d}{dh_i}\sigma(h_i)$. Typical choices of these non-Gaussian distributions have larger peaks near 0 than does the Gaussian distribution, so we can also see most implementations of ICA as learning sparse features.

Many variants of ICA are not generative models in the sense that we use the phrase. In this book, a generative model either represents $p(\mathbf{x})$ or can draw samples from it. Many variants of ICA only know how to transform between $\mathbf{x}$ and $\mathbf{h}$, but do not have any way of representing $p(\mathbf{h})$, and thus do not impose a distribution over $p(\mathbf{x})$. For example, many ICA variants aim to increase the sample kurtosis of $\mathbf{h} = \mathbf{W}^{-1}\mathbf{x}$, because high kurtosis indicates that $p(\mathbf{h})$ is non-Gaussian, but this is accomplished without explicitly representing $p(\mathbf{h})$. This is because ICA is more often used as an analysis tool for separating signals, rather than for generating data or estimating its density.

Just as PCA can be generalized to the nonlinear autoencoders described in chapter 14, ICA can be generalized to a nonlinear generative model, in which we use a nonlinear function f to generate the observed data. See Hyvärinen and Pajunen (1999) for the initial work on nonlinear ICA and its successful use with ensemble learning by Roberts and Everson (2001) and Lappalainen *et al.* (2000). Another nonlinear extension of ICA is the approach of **nonlinear independent components estimation**, or NICE (Dinh *et al.*, 2014), which stacks a series of invertible transformations (encoder stages) that have the property that the determinant of the Jacobian of each transformation can be computed efficiently. This makes it possible to compute the likelihood exactly and, like ICA, attempts to transform the data into a space where it has a factorized marginal distribution, but is more likely to succeed thanks to the nonlinear encoder. Because the encoder is associated with a decoder that is its perfect inverse, it is straightforward to generate samples from the model (by first sampling from $p(\mathbf{h})$ and then applying the decoder).

Another generalization of ICA is to learn groups of features, with statistical dependence allowed within a group but discouraged between groups (Hyvärinen and Hoyer, 1999; Hyvärinen *et al.*, 2001b). When the groups of related units are chosen to be non-overlapping, this is called **independent subspace analysis**. It is also possible to assign spatial coordinates to each hidden unit and form overlapping groups of spatially neighboring units. This encourages nearby units to learn similar features. When applied to natural images, this **topographic ICA** approach learns Gabor filters, such that neighboring features have similar orientation, location or frequency. Many different phase offsets of similar Gabor functions occur within each region, so that pooling over small regions yields translation invariance.

13.3 Slow Feature Analysis

Slow feature analysis (SFA) is a linear factor model that uses information from

time signals to learn invariant features (Wiskott and Sejnowski, 2002).

Slow feature analysis is motivated by a general principle called the slowness principle. The idea is that the important characteristics of scenes change very slowly compared to the individual measurements that make up a description of a scene. For example, in computer vision, individual pixel values can change very rapidly. If a zebra moves from left to right across the image, an individual pixel will rapidly change from black to white and back again as the zebra's stripes pass over the pixel. By comparison, the feature indicating whether a zebra is in the image will not change at all, and the feature describing the zebra's position will change slowly. We therefore may wish to regularize our model to learn features that change slowly over time.

The slowness principle predates slow feature analysis and has been applied to a wide variety of models (Hinton, 1989; Földiák, 1989; Mobahi *et al.*, 2009; Bergstra and Bengio, 2009). In general, we can apply the slowness principle to any differentiable model trained with gradient descent. The slowness principle may be introduced by adding a term to the cost function of the form

$$\lambda \sum_t L(f(\mathbf{x}^{(t+1)}), f(\mathbf{x}^{(t)})) \quad (13.7)$$

where λ is a hyperparameter determining the strength of the slowness regularization term, t is the index into a time sequence of examples, f is the feature extractor to be regularized, and L is a loss function measuring the distance between $f(\mathbf{x}^{(t)})$ and $f(\mathbf{x}^{(t+1)})$. A common choice for L is the mean squared difference.

Slow feature analysis is a particularly efficient application of the slowness principle. It is efficient because it is applied to a linear feature extractor, and can thus be trained in closed form. Like some variants of ICA, SFA is not quite a generative model per se, in the sense that it defines a linear map between input space and feature space but does not define a prior over feature space and thus does not impose a distribution $p(\mathbf{x})$ on input space.

The SFA algorithm (Wiskott and Sejnowski, 2002) consists of defining $f(\mathbf{x}; \boldsymbol{\theta})$ to be a linear transformation, and solving the optimization problem

$$\min_{\boldsymbol{\theta}} \mathbb{E}_t (f(\mathbf{x}^{(t+1)})_i - f(\mathbf{x}^{(t)})_i)^2 \quad (13.8)$$

subject to the constraints

$$\mathbb{E}_t f(\mathbf{x}^{(t)})_i = 0 \quad (13.9)$$

and

$$\mathbb{E}_t [f(\mathbf{x}^{(t)})_i^2] = 1. \quad (13.10)$$

The constraint that the learned features have zero mean is necessary to make the problem have a unique solution; otherwise we could add a constant to all feature values and obtain a different solution with equal value of the slowness objective. The constraint that the features have unit variance is necessary to prevent the pathological solution where all features collapse to 0. Like PCA, the SFA features are ordered, with the first feature being the slowest. To learn multiple features, we must also add the constraint

$$\forall i < j, \mathbb{E}_t[f(\mathbf{x}^{(t)})_i f(\mathbf{x}^{(t)})_j] = 0. \quad (13.11)$$

This specifies that the learned features must be linearly decorrelated from each other. Without this constraint, all of the learned features would simply capture the one slowest signal. One could imagine using other mechanisms, such as minimizing reconstruction error, to force the features to diversify, but this decorrelation mechanism admits a simple solution due to the linearity of SFA features. The SFA problem may be solved in closed form by a linear algebra package.

SFA is typically used to learn nonlinear features by applying a nonlinear basis expansion to $\mathbf{x}$ before running SFA. For example, it is common to replace $\mathbf{x}$ by the quadratic basis expansion, a vector containing elements $x_i x_j$ for all i and j . Linear SFA modules may then be composed to learn deep nonlinear slow feature extractors by repeatedly learning a linear SFA feature extractor, applying a nonlinear basis expansion to its output, and then learning another linear SFA feature extractor on top of that expansion.

When trained on small spatial patches of videos of natural scenes, SFA with quadratic basis expansions learns features that share many characteristics with those of complex cells in V1 cortex (Berkes and Wiskott, 2005). When trained on videos of random motion within 3-D computer rendered environments, deep SFA learns features that share many characteristics with the features represented by neurons in rat brains that are used for navigation (Franzius *et al.*, 2007). SFA thus seems to be a reasonably biologically plausible model.

A major advantage of SFA is that it is possible to theoretically predict which features SFA will learn, even in the deep, nonlinear setting. To make such theoretical predictions, one must know about the dynamics of the environment in terms of configuration space (e.g., in the case of random motion in the 3-D rendered environment, the theoretical analysis proceeds from knowledge of the probability distribution over position and velocity of the camera). Given the knowledge of how the underlying factors actually change, it is possible to analytically solve for the optimal functions expressing these factors. In practice, experiments with deep SFA applied to simulated data seem to recover the theoretically predicted functions.

This is in comparison to other learning algorithms where the cost function depends highly on specific pixel values, making it much more difficult to determine what features the model will learn.

Deep SFA has also been used to learn features for object recognition and pose estimation (Franzius *et al.*, 2008). So far, the slowness principle has not become the basis for any state of the art applications. It is unclear what factor has limited its performance. We speculate that perhaps the slowness prior is too strong, and that, rather than imposing a prior that features should be approximately constant, it would be better to impose a prior that features should be easy to predict from one time step to the next. The position of an object is a useful feature regardless of whether the object’s velocity is high or low, but the slowness principle encourages the model to ignore the position of objects that have high velocity.

13.4 Sparse Coding

Sparse coding (Olshausen and Field, 1996) is a linear factor model that has been heavily studied as an unsupervised feature learning and feature extraction mechanism. Strictly speaking, the term “sparse coding” refers to the process of inferring the value of $\mathbf{h}$ in this model, while “sparse modeling” refers to the process of designing and learning the model, but the term “sparse coding” is often used to refer to both.

Like most other linear factor models, it uses a linear decoder plus noise to obtain reconstructions of $\mathbf{x}$, as specified in equation 13.2. More specifically, sparse coding models typically assume that the linear factors have Gaussian noise with isotropic precision β :

$$p(\mathbf{x} \mid \mathbf{h}) = \mathcal{N}(\mathbf{x}; \mathbf{W}\mathbf{h} + \mathbf{b}, \frac{1}{\beta}\mathbf{I}). \quad (13.12)$$

The distribution $p(\mathbf{h})$ is chosen to be one with sharp peaks near 0 (Olshausen and Field, 1996). Common choices include factorized Laplace, Cauchy or factorized Student- t distributions. For example, the Laplace prior parametrized in terms of the sparsity penalty coefficient λ is given by

$$p(h_i) = \text{Laplace}(h_i; 0, \frac{2}{\lambda}) = \frac{\lambda}{4} e^{-\frac{1}{2}\lambda|h_i|} \quad (13.13)$$

and the Student- t prior by

$$p(h_i) \propto \frac{1}{(1 + \frac{h_i^2}{\nu})^{\frac{\nu+1}{2}}}. \quad (13.14)$$

Training sparse coding with maximum likelihood is intractable. Instead, the training alternates between encoding the data and training the decoder to better reconstruct the data given the encoding. This approach will be justified further as a principled approximation to maximum likelihood later, in section 19.3.

For models such as PCA, we have seen the use of a parametric encoder function that predicts $\mathbf{h}$ and consists only of multiplication by a weight matrix. The encoder that we use with sparse coding is not a parametric encoder. Instead, the encoder is an optimization algorithm, that solves an optimization problem in which we seek the single most likely code value:

$$\mathbf{h}^* = f(\mathbf{x}) = \arg \max_{\mathbf{h}} p(\mathbf{h} | \mathbf{x}). \quad (13.15)$$

When combined with equation 13.13 and equation 13.12, this yields the following optimization problem:

$$\arg \max_{\mathbf{h}} p(\mathbf{h} | \mathbf{x}) \quad (13.16)$$

$$= \arg \max_{\mathbf{h}} \log p(\mathbf{h} | \mathbf{x}) \quad (13.17)$$

$$= \arg \min_{\mathbf{h}} \lambda \|\mathbf{h}\|_1 + \beta \|\mathbf{x} - \mathbf{W}\mathbf{h}\|_2^2, \quad (13.18)$$

where we have dropped terms not depending on $\mathbf{h}$ and divided by positive scaling factors to simplify the equation.

Due to the imposition of an L^1 norm on $\mathbf{h}$, this procedure will yield a sparse $\mathbf{h}^*$ (See section 7.1.2).

To train the model rather than just perform inference, we alternate between minimization with respect to $\mathbf{h}$ and minimization with respect to $\mathbf{W}$. In this presentation, we treat β as a hyperparameter. Typically it is set to 1 because its role in this optimization problem is shared with λ and there is no need for both hyperparameters. In principle, we could also treat β as a parameter of the model and learn it. Our presentation here has discarded some terms that do not depend on $\mathbf{h}$ but do depend on β . To learn β , these terms must be included, or β will collapse to 0.

Not all approaches to sparse coding explicitly build a $p(\mathbf{h})$ and a $p(\mathbf{x} | \mathbf{h})$. Often we are just interested in learning a dictionary of features with activation values that will often be zero when extracted using this inference procedure.

If we sample $\mathbf{h}$ from a Laplace prior, it is in fact a zero probability event for an element of $\mathbf{h}$ to actually be zero. The generative model itself is not especially sparse, only the feature extractor is. Goodfellow *et al.* (2013d) describe approximate

inference in a different model family, the spike and slab sparse coding model, for which samples from the prior usually contain true zeros.

The sparse coding approach combined with the use of the non-parametric encoder can in principle minimize the combination of reconstruction error and log-prior better than any specific parametric encoder. Another advantage is that there is no generalization error to the encoder. A parametric encoder must learn how to map $\mathbf{x}$ to $\mathbf{h}$ in a way that generalizes. For unusual $\mathbf{x}$ that do not resemble the training data, a learned, parametric encoder may fail to find an $\mathbf{h}$ that results in accurate reconstruction or a sparse code. For the vast majority of formulations of sparse coding models, where the inference problem is convex, the optimization procedure will always find the optimal code (unless degenerate cases such as replicated weight vectors occur). Obviously, the sparsity and reconstruction costs can still rise on unfamiliar points, but this is due to generalization error in the decoder weights, rather than generalization error in the encoder. The lack of generalization error in sparse coding's optimization-based encoding process may result in better generalization when sparse coding is used as a feature extractor for a classifier than when a parametric function is used to predict the code. Coates and Ng (2011) demonstrated that sparse coding features generalize better for object recognition tasks than the features of a related model based on a parametric encoder, the linear-sigmoid autoencoder. Inspired by their work, Goodfellow *et al.* (2013d) showed that a variant of sparse coding generalizes better than other feature extractors in the regime where extremely few labels are available (twenty or fewer labels per class).

The primary disadvantage of the non-parametric encoder is that it requires greater time to compute $\mathbf{h}$ given $\mathbf{x}$ because the non-parametric approach requires running an iterative algorithm. The parametric autoencoder approach, developed in chapter 14, uses only a fixed number of layers, often only one. Another disadvantage is that it is not straight-forward to back-propagate through the non-parametric encoder, which makes it difficult to pretrain a sparse coding model with an unsupervised criterion and then fine-tune it using a supervised criterion. Modified versions of sparse coding that permit approximate derivatives do exist but are not widely used (Bagnell and Bradley, 2009).

Sparse coding, like other linear factor models, often produces poor samples, as shown in figure 13.2. This happens even when the model is able to reconstruct the data well and provide useful features for a classifier. The reason is that each individual feature may be learned well, but the factorial prior on the hidden code results in the model including random subsets of all of the features in each generated sample. This motivates the development of deeper models that can impose a non-

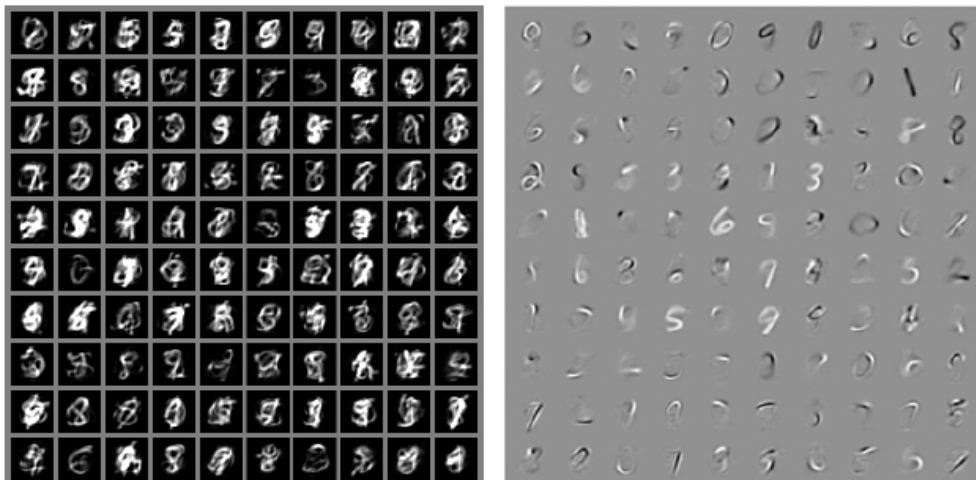


Figure 13.2: Example samples and weights from a spike and slab sparse coding model trained on the MNIST dataset. *(Left)* The samples from the model do not resemble the training examples. At first glance, one might assume the model is poorly fit. *(Right)* The weight vectors of the model have learned to represent penstrokes and sometimes complete digits. The model has thus learned useful features. The problem is that the factorial prior over features results in random subsets of features being combined. Few such subsets are appropriate to form a recognizable MNIST digit. This motivates the development of generative models that have more powerful distributions over their latent codes. Figure reproduced with permission from [Goodfellow *et al.* \(2013d\)](#).

factorial distribution on the deepest code layer, as well as the development of more sophisticated shallow models.

13.5 Manifold Interpretation of PCA

Linear factor models including PCA and factor analysis can be interpreted as learning a manifold ([Hinton *et al.*, 1997](#)). We can view probabilistic PCA as defining a thin pancake-shaped region of high probability—a Gaussian distribution that is very narrow along some axes, just as a pancake is very flat along its vertical axis, but is elongated along other axes, just as a pancake is wide along its horizontal axes. This is illustrated in figure 13.3. PCA can be interpreted as aligning this pancake with a linear manifold in a higher-dimensional space. This interpretation applies not just to traditional PCA but also to any linear autoencoder that learns matrices $\mathbf{W}$ and $\mathbf{V}$ with the goal of making the reconstruction of $\mathbf{x}$ lie as close to $\mathbf{x}$ as possible,

Let the encoder be

$$\mathbf{h} = f(\mathbf{x}) = \mathbf{W}^\top(\mathbf{x} - \boldsymbol{\mu}). \quad (13.19)$$

The encoder computes a low-dimensional representation of $\mathbf{h}$. With the autoencoder view, we have a decoder computing the reconstruction

$$\hat{\mathbf{x}} = g(\mathbf{h}) = \mathbf{b} + \mathbf{V}\mathbf{h}. \quad (13.20)$$

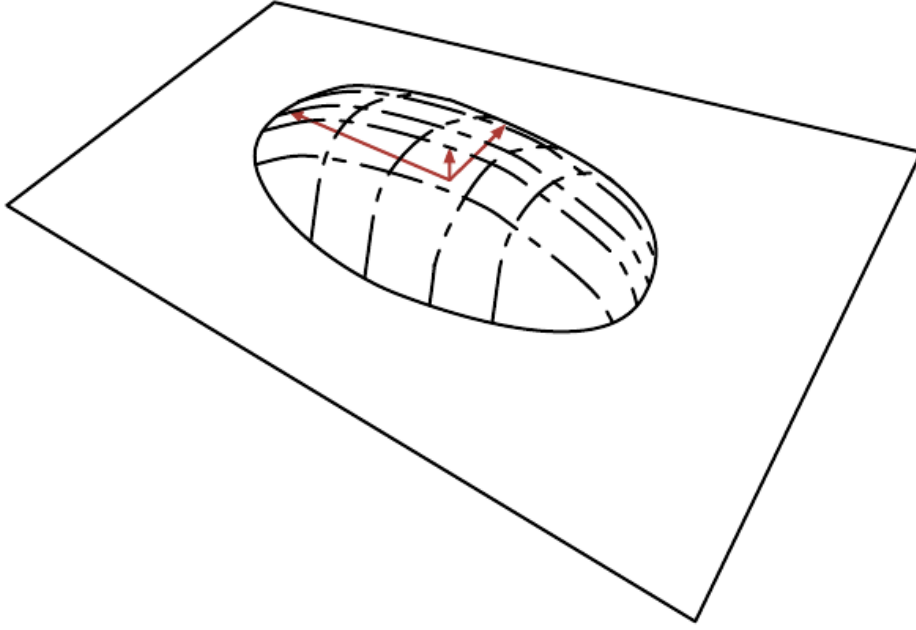


Figure 13.3: Flat Gaussian capturing probability concentration near a low-dimensional manifold. The figure shows the upper half of the “pancake” above the “manifold plane” which goes through its middle. The variance in the direction orthogonal to the manifold is very small (arrow pointing out of plane) and can be considered like “noise,” while the other variances are large (arrows in the plane) and correspond to “signal,” and a coordinate system for the reduced-dimension data.

The choices of linear encoder and decoder that minimize reconstruction error

$$\mathbb{E}[\|\mathbf{x} - \hat{\mathbf{x}}\|^2] \quad (13.21)$$

correspond to $\mathbf{V} = \mathbf{W}$, $\boldsymbol{\mu} = \mathbf{b} = \mathbb{E}[\mathbf{x}]$ and the columns of $\mathbf{W}$ form an orthonormal basis which spans the same subspace as the principal eigenvectors of the covariance matrix

$$\mathbf{C} = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top]. \quad (13.22)$$

In the case of PCA, the columns of $\mathbf{W}$ are these eigenvectors, ordered by the magnitude of the corresponding eigenvalues (which are all real and non-negative).

One can also show that eigenvalue λ_i of $\mathbf{C}$ corresponds to the variance of $\mathbf{x}$ in the direction of eigenvector $\mathbf{v}^{(i)}$. If $\mathbf{x} \in \mathbb{R}^D$ and $\mathbf{h} \in \mathbb{R}^d$ with $d < D$, then the

optimal reconstruction error (choosing $\boldsymbol{\mu}$, $\mathbf{b}$, $\mathbf{V}$ and $\mathbf{W}$ as above) is

$$\min \mathbb{E}[\|\mathbf{x} - \hat{\mathbf{x}}\|^2] = \sum_{i=d+1}^D \lambda_i. \quad (13.23)$$

Hence, if the covariance has rank d , the eigenvalues λ_{d+1} to λ_D are 0 and reconstruction error is 0.

Furthermore, one can also show that the above solution can be obtained by maximizing the variances of the elements of $\mathbf{h}$, under orthogonal $\mathbf{W}$, instead of minimizing reconstruction error.

Linear factor models are some of the simplest generative models and some of the simplest models that learn a representation of data. Much as linear classifiers and linear regression models may be extended to deep feedforward networks, these linear factor models may be extended to autoencoder networks and deep probabilistic models that perform the same tasks but with a much more powerful and flexible model family.